

Tugas 4 - Sistem Operasi



Nama : Putra Ganda Prayoga

Nim : 2410651057

Prodi Informatika

Fakultas Teknik

Universitas Muhammadiyah Jember

Experimen dengan Data berbeda

1.Buka Ubuntu Terminal.

2.Pastikan compiler GCC sudah terpasang jika belum jalankan ini:

`sudo apt update`

`udo apt install build-essential`

3.buat tiga file program:

1. FCFS $\rightarrow$ `fcfs.c`
2. SJF $\rightarrow$ `sjf.c`
3. Round Robin $\rightarrow$ `rr.c`

4.masukan kode tersebut disetiap file:

`#include <stdio.h>`

`int main() {`

`int n, i;`

`float avgWT = 0, avgTAT = 0;`

`printf("Masukkan jumlah proses: ");`

`scanf("%d", &n);`

`int at[n], bt[n], wt[n], tat[n], ct[n];`

`for (i = 0; i < n; i++) {`

`printf("Masukkan AT dan BT untuk P%d: ", i+1);`

`scanf("%d %d", &at[i], &bt[i]);`

`}`

```

int t = 0;

for (i = 0; i < n; i++) {
    if (t < at[i]) t = at[i];
    t += bt[i];
    ct[i] = t;
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    avgWT += wt[i];
    avgTAT += tat[i];
}

printf("\nProses\tAT\tBT\tCT\tTAT\tWT\n");

for (i = 0; i < n; i++)
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], ct[i], tat[i],
wt[i]);

printf("\nAverage Waiting Time = %.2f", avgWT/n);
printf("\nAverage Turnaround Time = %.2f\n", avgTAT/n);
return 0;
}

```

5. Tekan **ctrl + o** untuk simpan lalu enter dan terakhir **ctrl+x** untuk keluar

6. **Kompilasi FCFS compile dan jalankan:**

```
gcc fcfs.c -o fcfs
```

7. **Terakhir compile dan jalankan:**

```
./fcfs
```

8. **Masukan jumlahnya**

Masukkan jumlah proses: 3

Masukkan AT dan BT untuk P1: 0 10

Masukkan AT dan BT untuk P2: 1 5

Masukkan AT dan BT untuk P3: 2 8

8. **Hasilnya akan seperti berikut**

```
putra@DESKTOP-87VNBLO:~/Praktikum_Scheduling$ gcc fcfs.c -o fcfs
putra@DESKTOP-87VNBLO:~/Praktikum_Scheduling$ ./fcfs
Masukkan jumlah proses: 3
Masukkan AT dan BT untuk P1: 0 10
Masukkan AT dan BT untuk P2: 1 5
Masukkan AT dan BT untuk P3: 2 8

Proses  AT      BT      CT      TAT      WT
P1       0      10      10      10       0
P2       1       5      15      14       9
P3       2       8      23      21      13

Average Waiting Time = 7.33
Average Turnaround Time = 15.00
```

9.Kompilasi SJF compail dan jalankan kode berikut:

```
gcc sjf.c -o sjf
```

10.Terakhir compail dan jalankan:

```
./sjf
```

11.Masukan jumlahnya

Masukkan jumlah proses: 3

Masukkan AT dan BT untuk P1: 0 10

Masukkan AT dan BT untuk P2: 1 5

Masukkan AT dan BT untuk P3: 2 8

12.Hasilnya seperti berikut

```
putra@DESKTOP-87VNBLO:~/Praktikum_Scheduling$ ./sjf
Masukkan jumlah proses: 3
Masukkan AT dan BT untuk P1: 0 10
Masukkan AT dan BT untuk P2: 1 5
Masukkan AT dan BT untuk P3: 2 8

Proses  AT      BT      CT      TAT      WT
P1      0       10      10      10       0
P2      1        5      15      14       9
P3      2        8      23      21      13

Average Waiting Time = 7.33
Average Turnaround Time = 15.00
```

13.Kompilasi Kompilasi Round Robin compail dan jalankan kode berikut:

```
gcc rr.c -o rr
```

12.Terakhir compail dan jalankan:

```
./rr
```

14.Masukan jumlahnya

Masukkan jumlah proses: 3

Masukkan AT dan BT untuk P1: 0 10

Masukkan AT dan BT untuk P2: 1 5

Masukkan AT dan BT untuk P3: 2 8

15.Hasilnya seperti berikut

```
putra@DESKTOP-87VNBLO:~/Praktikum_Scheduling$ gcc rr.c -o rr
putra@DESKTOP-87VNBLO:~/Praktikum_Scheduling$ ./rr
Masukkan jumlah proses: 3
Masukkan AT dan BT untuk P1: 0 10
Masukkan AT dan BT untuk P2: 1 5
Masukkan AT dan BT untuk P3: 2 8

Proses  AT      BT      CT      TAT      WT
P1       0       10       10       10        0
P2       1        5       15       14        9
P3       2        8       23       21       13

Average Waiting Time = 7.33
Average Turnaround Time = 15.00
```

Analisis

1. FCFS (First Come First Serve)

- Sederhana dan mudah diimplementasikan.
- Kekurangan: bisa menyebabkan *convoy effect* (proses kecil menunggu proses besar selesai).
- Cocok untuk sistem *batch processing*.

2. SJF (Shortest Job First)

- Memberikan rata-rata waktu tunggu paling kecil jika burst time diketahui.
- Tidak adil bagi proses yang burst time-nya besar.
- Cocok untuk sistem dengan pekerjaan jangka pendek dan stabil.

3. RR (Round Robin)

- Menggunakan *time quantum* untuk membagi CPU secara bergiliran.
- Cocok untuk sistem *multitasking* dan interaktif.
- Kinerja tergantung pada ukuran *time quantum* (terlalu kecil → sering context switch, terlalu besar → seperti FCFS).

Kesimpulan

- Setiap algoritma memiliki kelebihan dan kekurangannya masing-masing tergantung pada kebutuhan sistem.
- **SJF** menghasilkan *Average Waiting Time* dan *Turnaround Time* terbaik untuk data yang burst time-nya bervariasi.
- **RR** paling adil untuk sistem multitasking karena setiap proses mendapat giliran CPU.
- **FCFS** paling mudah dipahami tetapi kurang efisien jika ada proses panjang di awal.